APCS 程式設計系列 · U05 · 實作中級 · C++ / Python 對照版

二維陣列與矩陣模擬 詳解

宣告與走訪（列 / 行）· 初始化陷阱 · 轉置與旋轉 90 度 · 方向陣列與邊界檢查 · 地圖模擬 · 螺旋走訪

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
A	C	C	C	B	B	D	B	B	A
11	**12**	**13**	**14**	**15**					
D	D	D	B	A					

Part A 程式識讀詳解

第 1 題 答案：(A)

- j 只跑到 i - 1，加到的是「對角線左下方」的格子：a[1][0]、a[2][0]、a[2][1]。
- 4 + 7 + 8 = 19。

【錯】看成右上方（2 + 3 + 6）會選 11；含對角線的話又不一樣，一定要列出 (i, j)。

第 2 題 答案：(C)

- 前面有 3 整列，每列 7 個：3 × 7 = 21。
- 再加上這一列前面的 4 個：21 + 4 = 25。

【錯】每列有幾個要看「行數」7，不是列數 5（用 5 會算成 19）。

第 3 題 答案：(C)

- 外層跑 j（行）、內層跑 i（列），是「行優先」：一行一行由上往下印。
- 第 0 行印 1 4，第 1 行印 2 5，第 2 行印 3 6。

第 4 題 答案：(C)

- s 是「第 j 行」由上到下的和：15、16、12、18。
- 最大的是第 3 行的 18。

【錯】當成算每一列的和，會得到最大是第 1 列的 22。

第 5 題 答案：(B)

- 沒有內層大括號時，依「列優先」一個一個填：第 0 列是 1 2 3，第 1 列是 4 5 和一個沒給的值。
- 初始化時沒給到的元素自動補 0，所以 a[1][0] = 4、a[1][2] = 0。

第 6 題 答案：(B)

- [[0] * 3] * 2 是把「同一個串列」放兩次，g[0] 和 g[1] 是同一個。
- 改 g[0][1] 就等於改 g[1][1]，印出 [0, 7, 0]。

【錯】以為兩列是各自獨立的，就會選 [0, 0, 0]。

第 7 題 答案：(D)

- (i, j) = (0, 1) 時交換一次，(i, j) = (1, 0) 時又交換一次，換回原樣。
- 每一對都被換兩次，矩陣完全沒變：a[0][1] = 2、a[1][0] = 4。

【錯】以為這是轉置而選 4 2。正確的原地轉置，內層要從 j = i + 1 開始。

第 8 題 答案：(B)

- 這是 n = 2 的順時針旋轉 b[j][n-1-i] = a[i][j]，b 會是 4 1 / 5 2 / 6 3。
- b[0][0] = 4、b[0][1] = 1、b[2][0] = 6。

第 9 題 答案：(B)

- 逆時針轉，原本「右上角」a[0][m-1] 會跑到新陣列的「左上角」b[0][0]。
- 代 i = 0、j = m-1 進 (B)：b[m-1-(m-1)][0] = b[0][0]，正確。(A) 是順時針、(C) 是轉置、(D) 是轉 180 度。

第 10 題 答案：(A)

- (2, 1) 的上方 (1, 1) = 1；下方 (3, 1) 出界跳過；左方 (2, 0) = 1；右方 (2, 2) = 0。
- 1 + 1 + 0 = 2。自己那一格不算。

第 11 題 答案：(D)

- dr = -1 時讀 a[-1][1]：Python 的 -1 代表「最後一列」，讀到 8，不會出錯。
- dr = 1 時讀 a[1][1] = 5。8 + 5 = 13。

【錯】以為出界會報錯，或以為會被略過。Python 負索引會「繞到另一邊」，所以一定要自己檢查邊界。

第 12 題 答案：(D)

- (0, 1) 在第一列，上面一排 3 格都出界。
- 剩下左右 2 格、下面一排 3 格，都在範圍內：5 格。

第 13 題 答案：(D)

- 算的是第 1 列（中間那列）每一格周圍 3 × 3 範圍內的 * 數。
- 第 0 格：(0,0) 和 (2,1) 共 2；第 1 格：(0,0)、(2,1) 共 2；第 2 格：(0,3)、(2,1) 共 2；第 3 格：只有 (0,3) 共 1。

第 14 題 答案：(B)

- 右、右走到 (0, 2)；第三個「右」會出界，留在原地。
- 下、下到 (2, 2)；左、左到 (2, 0)；上到 (1, 0)。

【錯】沒注意到第三個「右」被擋下來，整條路線都偏一格，會算成 1 1。

第 15 題 答案：(A)

- 方向依序是右、下、左、上；出界或走過就右轉。
- 往右 1 2 3，往下 6 9，往左 8 7，往上 4（1 走過了，右轉），往右 5：順時針螺旋。

Part B 程式實作參考解答

實作 1 照片旋轉

解題思路

- 轉 4 次會回到原樣，所以只要轉 k % 4 次（0 ~ 3 次）。
- 轉一次：開一個 m 列 n 行的新陣列 b，b[j][n-1-i] = a[i][j]，再把 b 當成新的 a。
- 每轉一次，列數和行數要對調。

【錯】k 到 10^9，真的轉 k 次會超時，一定要先取 % 4。

【錯】轉完忘了把 n、m 對調，非正方形的矩陣就會印錯（或越界）。

C++ 照片旋轉

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    long long k;
    cin >> n >> m >> k;
    vector<vector<int>> a(n, vector<int>(m));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) cin >> a[i][j];
    for (int t = 0; t < k % 4; t++) {
        vector<vector<int>> b(m, vector<int>(n));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                b[j][n - 1 - i] = a[i][j];
        a = b;
        swap(n, m);                 // 新的列數、行數
    }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cout << a[i][j] << (j + 1 < m ? ' ' : '\n');
}
```

Python

```python
import sys
data = sys.stdin.read().split()
n, m, k = int(data[0]), int(data[1]), int(data[2])
nums = list(map(int, data[3:3 + n * m]))
a = [nums[i * m:(i + 1) * m] for i in range(n)]
for _ in range(k % 4):
    b = [[0] * n for _ in range(m)]
    for i in range(n):
        for j in range(m):
            b[j][n - 1 - i] = a[i][j]
    a = b
    n, m = m, n                     # 新的列數、行數
for row in a:
    print(" ".join(map(str, row)))
```

時間複雜度：O(nm)（最多轉 3 次）

實作 2 踩地雷提示

解題思路

- 每一塊空地 (i, j)，檢查 r 從 i-1 到 i+1、c 從 j-1 到 j+1 的 9 格。
- 出界的格子跳過；自己是空地，所以算到自己也不影響。
- 地雷格直接輸出 *。

【錯】沒做邊界檢查：角落和邊上的格子會讀到陣列外面。

【錯】C++ 數字轉字元要寫 '0' + cnt；Python 的 g[-1] 會讀到最後一列，邊界要自己檢查。

C++ 踩地雷提示

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    for (int i = 0; i < n; i++) {
        string out = g[i];
        for (int j = 0; j < m; j++) {
            if (g[i][j] == '*') continue;
            int cnt = 0;
            for (int r = i - 1; r <= i + 1; r++)
                for (int c = j - 1; c <= j + 1; c++)
                    if (r >= 0 && r < n && c >= 0 && c < m
                        && g[r][c] == '*') cnt++;
            out[j] = '0' + cnt;
        }
        cout << out << "\n";
    }
}
```

Python

```python
import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
g = data[2:2 + n]
for i in range(n):
    line = []
    for j in range(m):
        if g[i][j] == "*":
            line.append("*")
            continue
        cnt = 0
        for r in range(i - 1, i + 2):
            for c in range(j - 1, j + 2):
                if 0 <= r < n and 0 <= c < m and g[r][c] == "*":
                    cnt += 1
        line.append(str(cnt))
    print("".join(line))
```

時間複雜度：O(9nm) = O(nm)

實作 3 掃地機器人

解題思路

- 把 U、R、D、L 對應到方向 0～3，用 dr、dc 方向陣列算下一格。
- 下一格出界或是 # 就跳過這個指令；否則移動過去。
- 開一個 visited 表格，第一次踩到某格才把計數加 1。

【錯】被擋住時是「這個指令不動」，不是整個程式停止，後面的指令還要繼續做。
【錯】掃過的格子要算「不同的」：同一格走兩次只算一次，而且出發點也要算。

C++ 掃地機器人

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m, r, c;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    string s;
    cin >> r >> c >> s;
    int dr[4] = {-1, 0, 1, 0}, dc[4] = {0, 1, 0, -1};
    string key = "URDL";
    vector<vector<bool>> seen(n, vector<bool>(m, false));
    seen[r][c] = true;
    int cnt = 1;
    for (char ch : s) {
        int k = key.find(ch);
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
        if (g[nr][nc] == '#') continue;
        r = nr; c = nc;
        if (!seen[r][c]) { seen[r][c] = true; cnt++; }
    }
    cout << r << " " << c << " " << cnt << "\n";
}
```

Python

```python
import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
g = data[2:2 + n]
r, c = int(data[2 + n]), int(data[3 + n])
s = data[4 + n]
move = {"U": (-1, 0), "R": (0, 1), "D": (1, 0), "L": (0, -1)}
seen = [[False] * m for _ in range(n)]
seen[r][c] = True
cnt = 1
for ch in s:
    nr, nc = r + move[ch][0], c + move[ch][1]
    if nr < 0 or nr >= n or nc < 0 or nc >= m or g[nr][nc] == "#":
        continue
    r, c = nr, nc
    if not seen[r][c]:
        seen[r][c] = True
        cnt += 1
print(r, c, cnt)
```

時間複雜度：O(nm + 指令長度)

實作 4 螺旋填數

解題思路

- 方向陣列照「右、下、左、上」排：dr = {0, 1, 0, -1}、dc = {1, 0, -1, 0}。
- 表格先全部設 0，0 代表還沒填。填完一格，先試著往目前方向走；出界或那一格不是 0，就 d = (d + 1) % 4 右轉。
- 總共填 n × m 次就結束。

【錯】只寫「出界才轉彎」會在第二圈撞上已經填過的格子，數字被覆蓋。

【錯】最後一格填完之後，「下一格」可能是出界的位置，但迴圈已經結束、不會再讀它，所以不會出錯；不要多跑一圈。

C++ 螺旋填數

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> a(n, vector<int>(m, 0));
    int dr[4] = {0, 1, 0, -1}, dc[4] = {1, 0, -1, 0};
    int r = 0, c = 0, d = 0;
    for (int v = 1; v <= n * m; v++) {
        a[r][c] = v;
        int nr = r + dr[d], nc = c + dc[d];
        bool out = nr < 0 || nr >= n || nc < 0 || nc >= m;
        if (out || a[nr][nc] != 0) {       // 撞牆或填過：右轉
            d = (d + 1) % 4;
            nr = r + dr[d]; nc = c + dc[d];
        }
        r = nr; c = nc;
    }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cout << a[i][j] << (j + 1 < m ? ' ' : '\n');
}
```

Python

```python
n, m = map(int, input().split())
a = [[0] * m for _ in range(n)]
dr, dc = [0, 1, 0, -1], [1, 0, -1, 0]
r = c = d = 0
for v in range(1, n * m + 1):
    a[r][c] = v
    nr, nc = r + dr[d], c + dc[d]
    out = nr < 0 or nr >= n or nc < 0 or nc >= m
    if out or a[nr][nc] != 0:           # 撞牆或填過：右轉
        d = (d + 1) % 4
        nr, nc = r + dr[d], c + dc[d]
    r, c = nr, nc
for row in a:
    print(" ".join(map(str, row)))
```

時間複雜度：O(nm)

實作 5 島嶼的面積與海岸線

解題思路

- 走訪每一格，是 # 就面積加 1。
- 再用四方向的方向陣列看它的鄰居：出界或是 .，海岸線就加 1。
- 中間有湖（被陸地圍住的海）時，湖邊也算海岸線，這個寫法會自動算進去。

【錯】斜對角不算相鄰，這題只看上下左右四個方向。

【錯】鄰居出界要先判斷；C++ 會讀到陣列外，Python 的 g[-1] 會讀到地圖另一邊。

C++ 島嶼的面積與海岸線

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    long long area = 0, coast = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (g[i][j] != '#') continue;
            area++;
            for (int k = 0; k < 4; k++) {
                int r = i + dr[k], c = j + dc[k];
                if (r < 0 || r >= n || c < 0 || c >= m) coast++;
                else if (g[r][c] == '.') coast++;
            }
        }
    cout << area << " " << coast << "\n";
}
```

Python

```python
import sys
data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
g = data[2:2 + n]
dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]
area = coast = 0
for i in range(n):
    for j in range(m):
        if g[i][j] != "#":
            continue
        area += 1
        for dr, dc in dirs:
            r, c = i + dr, j + dc
            if r < 0 or r >= n or c < 0 or c >= m or g[r][c] == ".":
                coast += 1
print(area, coast)
```

時間複雜度：O(4nm) = O(nm)

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
二維陣列走訪	A1、A2、A3、A4		綠 黃 紅	講義第 1 段（列優先、行優先）
初始化與共用陷阱	A5、A6		綠 黃 紅	講義第 2 段
轉置與旋轉	A7、A8、A9、B1		綠 黃 紅	講義第 3 段（公式表）
方向陣列與邊界	A10、A11、A12、B5		綠 黃 紅	講義第 4 段
地圖模擬	A13、A14、B2、B3		綠 黃 紅	講義第 5 段
螺旋走訪	A15、B4		綠 黃 紅	講義第 6 段